



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Core Platform & Build: 3.5 → 4.0

Twenty platform changes underneath everything

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything THIS GUIDE
The Big Jobs: L-Impact Changes	The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

21 CHANGES

13 x TIER 1 - WON'T BUILD

4 x TIER 2 - WON'T START

4 x TIER 3 - BEHAVES DIFFERENTLY

TIER	CHANGE	IMPACT	AUTO
1	AOP Starter Renamed to spring-boot-starter-aspectj	S	•
1	EnvironmentPostProcessor Relocated to org.springframework.boot	S	•
1	BootstrapRegistry Relocated to org.springframework.boot.bootstrap	S	•
1	Classic Uber-Jar Loader Removed	S	•
1	GraalVM 25 for Native Image Builds	M	•
1	Gradle 8.14+ / Gradle 9 Required	S	•
1	ListenableFuture Removed	M	•
1	Manual AOT Execution Blocks Now Redundant	S	•
1	Maven Plugin Version Alignment for AOT	S	Y
1	PropertyMapper.alwaysApplyingWhenNonNull() Removed	S	•
1	@PropertyMapping Relocated to test.context	S	•
1	spring-jcl Removed	S	Y
1	spring-retry Removed from BOM	S	•
2	Deprecated RestTemplateBuilder APIs Removed	S	•
2	javax.annotation Removed	S	Y
2	javax.inject Removed	S	Y
2	Modular Auto-Configuration	M	•
3	Logback Default Charset Changed to UTF-8	S	•
3	@NullMarked Default — IDE/Compiler Null Safety	M	•
3	maxAttempts Now Means Retries, Not Total Attempts	M	•
3	@Retryable + @Transactional AOP Ordering Change	M	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

Maven Plugin Version Alignment for AOT

spring-jcl Removed

javax.annotation Removed

javax.inject Removed

THE HUMAN LIST

Read the card. Check your code. Decide.

AOP Starter Renamed to spring-boot-starter-aspectj

EnvironmentPostProcessor Relocated to
org.springframework.boot

BootstrapRegistry Relocated to
org.springframework.boot.bootstrap

Classic Uber-Jar Loader Removed

GraalVM 25 for Native Image Builds

Gradle 8.14+ / Gradle 9 Required

ListenableFuture Removed

Manual AOT Execution Blocks Now Redundant

PropertyMapper.alwaysApplyingWhenNonNull() Removed

@PropertyMapping Relocated to test.context

spring-retry Removed from BOM

Deprecated RestTemplateBuilder APIs Removed

Modular Auto-Configuration

Logback Default Charset Changed to UTF-8

@NullMarked Default — IDE/Compiler Null Safety

maxAttempts Now Means Retries, Not Total Attempts

@Retryable + @Transactional AOP Ordering Change

TIER 1

Tier 1 - Won't Build

13 cards

1. AOP Starter Renamed to spring-boot-starter-aspectj
2. EnvironmentPostProcessor Relocated to org.springframework.boot
3. BootstrapRegistry Relocated to org.springframework.boot.bootstrap
4. Classic Uber-Jar Loader Removed
5. GraalVM 25 for Native Image Builds
6. Gradle 8.14+ / Gradle 9 Required
7. ListenableFuture Removed
8. Manual AOT Execution Blocks Now Redundant
9. Maven Plugin Version Alignment for AOT
10. PropertyMapper.alwaysApplyingWhenNonNull() Removed
11. @PropertyMapping Relocated to test.context
12. spring-jcl Removed
13. spring-retry Removed from BOM

AOP Starter Renamed to spring-boot-starter-aspectj

Tier 1 - Won't Build

Core Platform & Build

Impact: S

spring-boot-starter-aop is no longer in the Spring Boot 4.0 BOM. Replace it with spring-boot-starter-aspectj.

WHAT CHANGED

spring-boot-starter-aop has been removed from the Spring Boot BOM. The replacement, spring-boot-starter-aspectj, provides the same AspectJ weaving capabilities.

WHY

Spring AOP (interface-proxy and CGLIB-proxy based AOP) works without any starter. The old name implied you needed this starter for any AOP, when you only need it for AspectJ load-time or compile-time weaving. The rename makes the distinction clear.

SCOPE CHECK

Search all pom.xml and build.gradle files for spring-boot-starter-aop. Multi-module projects may declare it in several places.

WHAT YOU'LL SEE

```
$ mvn validate
[ERROR] 'dependencies.dependency.version' for
org.springframework.boot:spring-boot-starter-aop:jar is missing.
```

THE FIX

pom.xml dependency

BEFORE

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

HOW TO FIX

Replace the starter in pom.xml / build.gradle.

Find spring-boot-starter-aop in all build files and replace with spring-boot-starter-aspectj. No code changes needed: the classpath content is equivalent.

WATCH OUT

If you only use Spring's proxy-based AOP (`@Aspect` beans, `@Transactional` , etc.) you may not need this starter at all. Proxy-based AOP is included in `spring-boot-starter` transitively. The `aspectj` starter is only required if you use AspectJ weaving agents.

FURTHER INFO

This is a T1B failure (artifact no longer in the BOM): the build fails at dependency resolution, before any source compiles.

spring-break module: aop-starter-rename

Spring Boot 4.0 Migration Guide

EnvironmentPostProcessor Relocated to org.springframework.boot

Tier 1 - Won't Build

Core Platform & Build

Impact: S

EnvironmentPostProcessor moved from org.springframework.boot.env to org.springframework.boot. The spring.factories registration key must also change.

WHAT CHANGED

EnvironmentPostProcessor moved from org.springframework.boot.env to org.springframework.boot.

WHY

EnvironmentPostProcessor is used across many contexts, so it moved to the root package for discoverability.

SCOPE CHECK

Search for org.springframework.boot.env.EnvironmentPostProcessor across all Java/Kotlin sources and META-INF registration files.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/EnvironmentPostProcessorUsage.java:[3,43]
error: package org.springframework.boot.env does not contain EnvironmentPostProcessor
```

THE FIX

EnvironmentPostProcessor import

BEFORE

```
import org.springframework.boot.env.EnvironmentPostProcessor;
```

AFTER

```
import org.springframework.boot.EnvironmentPostProcessor;
```

HOW TO FIX

Update imports.

Find all usages of org.springframework.boot.env.EnvironmentPostProcessor and replace with org.springframework.boot.EnvironmentPostProcessor. A pure package move: no method signatures or behaviour changed.

Search spring.factories / AutoConfiguration.imports.

If you register an EnvironmentPostProcessor in META-INF/spring.factories or META-INF/spring/org.springframework.boot.env.EnvironmentPostProcessor.imports, update the key to org.springframework.boot.EnvironmentPostProcessor.

WATCH OUT

The spring.factories key for EnvironmentPostProcessor must also be updated. With the old key your processor is skipped at startup: no error, your environment customisation never runs.

FURTHER INFO

Affects code that customises early environment configuration. Verified in the same spring-break test module as the companion BootstrapRegistry relocation: the two moves go in opposite directions and are otherwise unrelated.

spring-break module: bootstrap-registry-relocated Spring Boot 4.0 Migration Guide

BootstrapRegistry Relocated to org.springframework.boot.bootstrap

Tier 1 - Won't Build

Core Platform & Build

Impact: S

BootstrapRegistry and ConfigurableBootstrapContext moved to org.springframework.boot.bootstrap. The old import path is gone.

WHAT CHANGED

BootstrapRegistry and ConfigurableBootstrapContext moved from org.springframework.boot to org.springframework.boot.bootstrap.

WHY

Spring Boot 4.0 groups its infrastructure by concern: bootstrap lifecycle classes got their own subpackage.

SCOPE CHECK

Search for org.springframework.boot.BootstrapRegistry and org.springframework.boot.ConfigurableBootstrapContext across all Java/Kotlin sources.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BootstrapRegistryUsage.java:[3,47]
error: package org.springframework.boot does not contain BootstrapRegistry
```

THE FIX

BootstrapRegistry import

BEFORE

```
import org.springframework.boot.BootstrapRegistry;
```

AFTER

```
import org.springframework.boot.bootstrap.BootstrapRegistry;
```

ConfigurableBootstrapContext import

BEFORE

```
import org.springframework.boot.ConfigurableBootstrapContext;
```

AFTER

```
import org.springframework.boot.bootstrap.ConfigurableBootstrapContext;
```

HOW TO FIX

Update imports.

Find all usages of org.springframework.boot.BootstrapRegistry and org.springframework.boot.ConfigurableBootstrapContext and replace with the .bootstrap subpackage. Both are pure package moves: no method signatures or behaviour changed.

FURTHER INFO

Affects code that customises early application startup. Verified in the same spring-break test module as the companion EnvironmentPostProcessor relocation: the two moves go in opposite directions and are otherwise unrelated.

spring-break module: bootstrap-registry-relocated Spring Boot 4.0 Migration Guide

Classic Uber-Jar Loader Removed

Tier 1 - Won't Build

Core Platform & Build

Impact: S

The `CLASSIC` uber-jar loader option is removed in Boot 4.0: the Maven/Gradle plugin fails the build if it is configured. Remove the setting.

WHAT CHANGED

The `CLASSIC` loader implementation, the original Spring Boot uber-jar format, was removed from the Maven and Gradle plugins. Any build file with `loaderImplementation=CLASSIC` fails at the packaging phase. The default nested-jar loader (introduced in Boot 3.2) is the only supported format.

WHY

The classic loader was deprecated in Spring Boot 3.2 when the nested-jar loader arrived; Boot 4.0 removes it. The new loader offers better compatibility with AOT compilation, virtual threads, and GraalVM native image tooling.

SCOPE CHECK

Search build files for `CLASSIC` and `loaderImplementation`. Check `pom.xml`, parent POMs, and any Gradle build scripts that configure the Spring Boot plugin.

WHAT YOU'LL SEE

```
<!-- pom.xml with CLASSIC loader -->
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <loaderImplementation>CLASSIC</loaderImplementation>
  </configuration>
</plugin>

// Boot 4.0 build error:
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin
...CLASSIC loader implementation is no longer supported
```

THE FIX

Maven — remove CLASSIC loader configuration

BEFORE

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <loaderImplementation>CLASSIC</loaderImplementation>
  </configuration>
</plugin>
```

AFTER

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <!-- No loaderImplementation needed - default is the only option -->
</plugin>
```

Gradle — remove loaderImplementation property

BEFORE

```
tasks.named("bootJar") {
  loaderImplementation = LoaderImplementation.CLASSIC
}
```

AFTER

```
// No loaderImplementation configuration needed
```

HOW TO FIX

Remove the CLASSIC loader configuration.

Delete `<loaderImplementation>CLASSIC</loaderImplementation>` from the Maven plugin configuration, or remove `loaderImplementation = LoaderImplementation.CLASSIC` from the Gradle boot jar task. The build then defaults to the modern nested-jar loader.

VERIFY

Application jar is built successfully and starts correctly after removing CLASSIC loader configuration from the build file

WATCH OUT

The new nested-jar loader changes the path structure inside the uber jar. Tools that inspect the jar directly (custom launch scripts, Docker entrypoints referencing internal paths) may need updates to reflect the `BOOT-INF/` layout rather than the classic flat layout.

FURTHER INFO

The plugin rejects the `CLASSIC` value with an explicit error rather than ignoring it, so the failure is loud and easy to find.

GraalVM 25 for Native Image Builds

Tier 1 - Won't Build

Core Platform & Build

Impact: M

Spring Boot 4.0 requires GraalVM 25 for native image compilation. Older GraalVM versions produce build failures or runtime crashes.

WHAT CHANGED

Spring Boot 4.0 targets GraalVM 25 (based on JDK 25) for native image builds. The `native-maven-plugin` and `org.graalvm.buildtools.native` Gradle plugin must be updated to version 0.10.6+. Older GraalVM versions (22.x, 23.x) are no longer compatible with the reachability metadata shipped in starters.

WHY

GraalVM 25 introduced a new metadata format and improved closed-world analysis. Spring's reachability metadata was rewritten to use these features, making it incompatible with older native-image compilers.

SCOPE CHECK

Search for `native-maven-plugin` and `org.graalvm.buildtools.native` in build files. Check CI pipelines for `GRAALVM_HOME` or `native-image` version references.

WHAT YOU'LL SEE

```
$ mvn -Pnative native:compile
[ERROR] Error: Unsupported native-image version: 22.3.5
[ERROR] Spring Boot 4.0 requires GraalVM 25.0 or later.
[ERROR] Please upgrade your GraalVM installation.
[ERROR]
[ERROR] -> [Help 1]
[ERROR] org.graalvm.buildtools:native-maven-plugin:0.10.6:compile failed
```

THE FIX

pom.xml — native build tools plugin

BEFORE

```
<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
  <version>0.9.28</version>
</plugin>
```

AFTER

```
<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
  <version>0.10.6</version>
</plugin>
```

build.gradle — GraalVM plugin

BEFORE

```
plugins {  
    id 'org.graalvm.buildtools.native' version '0.9.28'  
}
```

AFTER

```
plugins {  
    id 'org.graalvm.buildtools.native' version '0.10.6'  
}
```

JAVA_HOME / GRAALVM_HOME

BEFORE

```
export GRAALVM_HOME=/opt/graalvm-ce-java17-22.3.5
```

AFTER

```
export GRAALVM_HOME=/opt/graalvm-jdk-25+35
```

HOW TO FIX

Install GraalVM 25.

Download GraalVM 25 from graalvm.org and set `GRAALVM_HOME`. If using SDKMAN: `sdk install java 25-graal`.

Update native build tools.

Bump the `native-maven-plugin` or Gradle `org.graalvm.buildtools.native` plugin to 0.10.6+. The parent POM manages this if you inherit from `spring-boot-starter-parent`.

WATCH OUT

GraalVM 25 is based on JDK 25, but the native image it produces still runs on any compatible OS. Don't confuse the build-time JDK requirement with the runtime requirement.

If you use custom reachability metadata in `META-INF/native-image/`, verify it still works. The metadata format added new fields in GraalVM 25 and some older entries may be silently ignored.

FURTHER INFO

Applies only to native image builds via the `spring-boot-starter-parent` native profile; standard JVM deployments are unaffected.

Spring Boot 4.0 Migration Guide

Gradle 8.14+ / Gradle 9 Required

Tier 1 - Won't Build

Spring Boot 4.0's Gradle plugin requires Gradle 8.14 or later. Older Gradle wrappers fail immediately at configuration time.

Core Platform & Build

Impact: S

WHAT CHANGED

The Spring Boot Gradle plugin now requires Gradle 8.14 as a minimum. Gradle 9.0 is fully supported and recommended for new projects.

WHY

Gradle 8.14 introduced configuration cache stability and improved dependency resolution APIs that the Spring Boot plugin now relies on. Supporting older Gradle versions held back build performance improvements.

SCOPE CHECK

Check `gradle/wrapper/gradle-wrapper.properties` for the current Gradle version. Also check CI caches and any shared Gradle distributions. Multi-module projects only need one wrapper update.

WHAT YOU'LL SEE

```
$ ./gradlew build
FAILURE: Build failed with an exception.
* Where:
Build file '/project/build.gradle' line: 3
* What went wrong:
An exception occurred applying plugin request [id: 'org.springframework.boot', version: '4.0.0']
> Failed to apply plugin 'org.springframework.boot'.
   > Spring Boot plugin requires Gradle 8.14+. Found: 8.5.
```

THE FIX

gradle/wrapper/gradle-wrapper.properties

BEFORE

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.5-bin.zip
```

AFTER

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.14-bin.zip
```

Alternative: jump to Gradle 9

BEFORE

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.5-bin.zip
```

AFTER

```
distributionUrl=https\://services.gradle.org/distributions/gradle-9.0-bin.zip
```

HOW TO FIX

Update the Gradle wrapper.

Run `./gradlew wrapper --gradle-version 8.14` (or `9.0`) to update the wrapper in place. Commit the updated wrapper files.

Check plugin compatibility.

After upgrading Gradle, run a build to verify all other plugins are compatible. The [Gradle upgrade guide](#) lists deprecations to address.

WATCH OUT

If you're on Gradle 7.x, you can't jump straight to 8.14. Gradle deprecation cycles mean you need to fix warnings at each major version boundary. Plan for a staged upgrade: 7.x to 8.0, then 8.0 to 8.14+.

The Gradle configuration cache is now stable and on by default in Gradle 9. If your build scripts use project-level state at execution time, you'll hit configuration cache errors after upgrading.

FURTHER INFO

Maven builds are unaffected.

Spring Boot 4.0 Migration Guide

ListenableFuture Removed

Tier 1 - Won't Build

Spring Framework removed ListenableFuture entirely. All async code using it must switch to CompletableFuture.

Core Platform & Build

Impact: M

WHAT CHANGED

`ListenableFuture`, `ListenableFutureCallback`, `ListenableFutureTask`, and related classes in `org.springframework.util.concurrent` were deleted from Spring Framework 7.0. All Spring APIs that previously returned `ListenableFuture` now return `java.util.concurrent.CompletableFuture`.

WHY

`ListenableFuture` predated Java 8. Once `CompletableFuture` arrived in the JDK, maintaining a parallel abstraction added complexity with no benefit.

SCOPE CHECK

Search for `ListenableFuture`, `ListenableFutureCallback`, and `AsyncResult`. Also check for `addCallback()` calls: these usage sites need rewriting even if the declaration is in a library.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/service/AsyncService.java:[4,49]
error: cannot find symbol
  symbol:   class ListenableFuture
  location: package org.springframework.util.concurrent
[ERROR] /src/main/java/com/example/service/AsyncService.java:[5,49]
error: cannot find symbol
  symbol:   class ListenableFutureCallback
  location: package org.springframework.util.concurrent
```

THE FIX

Async method return type

BEFORE

```
import org.springframework.util.concurrent.ListenableFuture;

@Async
public ListenableFuture<String> fetchData() {
    return AsyncResult.forValue(doWork());
}
```

AFTER

```
import java.util.concurrent.CompletableFuture;

@Async
public CompletableFuture<String> fetchData() {
    return CompletableFuture.completedFuture(doWork());
}
```

BEFORE

```

ListenableFuture<String> future = asyncService.fetchData();
future.addCallback(
    result -> log.info("Success: {}", result),
    ex -> log.error("Failed", ex)
);

```

AFTER

```

CompletableFuture<String> future = asyncService.fetchData();
future.whenComplete((result, ex) -> {
    if (ex != null) log.error("Failed", ex);
    else log.info("Success: {}", result);
});

```

HOW TO FIX**Replace with CompletableFuture.**

Change return types from `ListenableFuture<T>` to `CompletableFuture<T>`. Replace `AsyncResult.forValue()` with `CompletableFuture.completedFuture()`. Replace `addCallback()` with `whenComplete()` or `thenAccept()`.

Use OpenRewrite.

The `org.openrewrite.java.spring.framework.MigrateSpringAssert` recipe set includes `ListenableFuture` migration.

WATCH OUT

`AsyncResult` was also removed. If you used `new AsyncResult<>(value)` or `AsyncResult.forValue()`, both must change to `CompletableFuture.completedFuture()`.

If you used `ListenableFuture` in a messaging listener (e.g., Spring Kafka's `KafkaTemplate.send()`), the return type changed upstream too. Check your Kafka and AMQP send-and-confirm patterns.

FURTHER INFO

Deprecated in Spring Framework 6.0, removed in 7.0 (tracked in [spring-framework#33808](#)). Spring Kafka and Spring AMQP return types that previously used `ListenableFuture` changed too.

[Spring-Break Demo](#) [Spring Framework ListenableFuture removal](#)

Manual AOT Execution Blocks Now Redundant

Tier 1 - Won't Build

Core Platform & Build

Impact: S

The Boot 4.0 parent POM binds the AOT goals automatically. Explicit `process-aot` execution blocks left over from 3.x make the goal run twice, or run with stale configuration that no longer matches the plugin.

WHAT CHANGED

In Boot 3.x, running AOT processing usually meant declaring an explicit `<execution>` block for the `process-aot` and `process-test-aot` goals. In 4.0 the parent POM binds these goals automatically, so a leftover manual block declares the goal a second time and applies configuration written for the 3.x goal.

WHY

AOT processing runs by default in Boot 4.0, even for non-native builds, so the build lifecycle now owns the goal binding. Manual blocks were only ever a workaround for AOT being opt-in.

SCOPE CHECK

Search all POM files for `process-aot` and `process-test-aot`. Any hit inside an `<execution>` block in a module that inherits `spring-boot-starter-parent` is redundant.

WHAT YOU'LL SEE

```
$ mvn package
[INFO] --- spring-boot-maven-plugin:4.0.0:process-aot (process-aot) @ my-service ---
[INFO] --- spring-boot-maven-plugin:4.0.0:process-aot (default-process-aot) @ my-service ---
...
[ERROR] Failed to execute goal ...:process-aot (process-aot) on project my-service:
    configuration parameter no longer supported by the 4.0 goal
---
Symptoms vary: duplicated AOT runs, longer builds, or a hard failure
when the old execution carries configuration the 4.0 goal rejects.
```

THE FIX

pom.xml — remove explicit AOT execution if using parent POM

BEFORE

```
<execution>
  <id>process-aot</id>
  <goals><goal>process-aot</goal></goals>
</execution>
```

AFTER

```
<!-- AOT goals are now bound automatically by the parent POM -->
```

HOW TO FIX

Delete manual AOT execution blocks.

Remove `<execution>` blocks for `process-aot` and `process-test-aot` from every POM that inherits the parent. Keep any non-AOT configuration by moving it to the plugin's top-level `<configuration>` section.

Not using the parent POM?

If you build from the BOM without `spring-boot-starter-parent`, you still own the goal binding. Keep the execution block but review its configuration against the 4.0 goal's parameters.

WATCH OUT

A duplicated goal that happens to succeed still doubles your AOT processing time; slow builds are the quiet version of this break.

CI pipelines that call `mvn spring-boot:process-aot` directly are also redundant under the parent POM and can conflict with the automatic binding.

FURTHER INFO

Split out from the plugin version alignment break: a project with a correctly managed plugin version can still carry stale execution blocks, and vice versa. See also: [maven-aot-plugin](#), [graalvm-25](#).

[Spring Boot 4.0 Migration Guide](#)

Maven Plugin Version Alignment for AOT

Tier 1 - Won't Build

Core Platform & Build

Impact: S

OpenRewrite

WHAT CHANGED

The `spring-boot-maven-plugin` must match the Spring Boot version exactly. In 4.0 the AOT processing API changed: the `process-aot` and `process-test-aot` goals use new generation context methods. A plugin version mismatch causes `NoSuchMethodError` at build time.

WHY

AOT processing was refactored for GraalVM 25 compatibility and to support the new Spring Framework 7.0 code generation model. The plugin version must be kept in lock-step with the framework.

SCOPE CHECK

Search for `spring-boot-maven-plugin` across all POM files and check for explicit `<version>` tags. Multi-module projects need every module's POM checked.

WHAT YOU'LL SEE

```
$ mvn spring-boot:process-aot
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot
on project my-service: Execution default of goal
org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot failed:
An API incompatibility was encountered while executing
org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot:
java.lang.NoSuchMethodError:
'void org.springframework.aot.generate.GenerationContext.<init>(java.lang.ClassLoader)'
```

THE FIX

pom.xml — plugin version

BEFORE

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <version>3.4.1</version>
</plugin>
```

AFTER

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <version>4.0.0</version>
</plugin>
```

HOW TO FIX

Align the plugin version with Spring Boot.

If you inherit from `spring-boot-starter-parent`, the plugin version is managed automatically: remove any explicit `<version>` tag. If you use a BOM without the parent POM, set the plugin version to `4.0.0` explicitly.

WATCH OUT

If you're not using native images, AOT processing is still enabled by default in 4.0 for startup optimisation. You may hit this error even if you never intentionally configured AOT.

Parent POM version and plugin version must match. If your `<parent>` points to 4.0.0 but a plugin override still says 3.x, the build will fail with cryptic reflection errors.

FURTHER INFO

AOT processing runs by default in Boot 4.0 even for non-native builds. Stale manual AOT `<execution>` blocks are a separate break: see [maven-aot-execution-blocks](#). See also: [graalvm-25](#).

Spring Boot 4.0 Migration Guide

PropertyMapper.alwaysApplyingWhenNonNull() Removed

Tier 1 - Won't Build

Core Platform & Build

Impact: S

PropertyMapper.alwaysApplyingWhenNonNull() is removed because skipping null values is now the default. Just delete the call.

WHAT CHANGED

PropertyMapper.alwaysApplyingWhenNonNull() is gone. Its behaviour, skipping mappings when the source value is `null`, is now the default for `PropertyMapper`, so calling it was a no-op from 4.0 onward.

WHY

The non-null default was the right behaviour for virtually all use cases. Requiring an explicit opt-in call was boilerplate with no practical reason to exist. Making it the default and removing the method simplifies the API.

SCOPE CHECK

Search for `alwaysApplyingWhenNonNull` across all Java/Kotlin sources. This typically appears in autoconfiguration classes or custom configuration adapters that mirror Spring Boot's internal style.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/PropertyMapperUsage.java:[8,47]
error: cannot find symbol
  symbol:   method alwaysApplyingWhenNonNull()
  location: class PropertyMapper
```

THE FIX

Remove the method call — the behaviour is now default

BEFORE

```
PropertyMapper map = PropertyMapper.get().alwaysApplyingWhenNonNull();
```

AFTER

```
PropertyMapper map = PropertyMapper.get();
```

HOW TO FIX

Delete the alwaysApplyingWhenNonNull() call.

Remove `.alwaysApplyingWhenNonNull()` from the chain. The mapper behaves identically on Boot 4.0 without it. If you explicitly want to map null values, use `.always()` on individual mappings.

WATCH OUT

If you were relying on the absence of `alwaysApplyingWhenNonNull()` to intentionally map null values through, that behaviour has also changed: nulls are now skipped by default. Add `.always()` on the specific mappings that need to pass nulls.

FURTHER INFO

PropertyMapper is an internal Spring Boot utility, used mostly in auto-configuration classes to map configuration properties onto builder objects.

@PropertyMapping Relocated to test.context

Tier 1 - Won't Build

Core Platform & Build

Impact: S

@PropertyMapping moved from `org.springframework.boot.test.autoconfigure.properties` to `org.springframework.boot.test.context`. Old import doesn't compile.

WHAT CHANGED

`@PropertyMapping` moved from `org.springframework.boot.test.autoconfigure.properties` to `org.springframework.boot.test.context`. Behaviour is unchanged.

WHY

Spring Boot 4.0 consolidated test-context infrastructure into a more coherent package hierarchy. The separate `test.autoconfigure.properties` package was collapsed into `test.context`.

SCOPE CHECK

Search for `@PropertyMapping` and the old import path in your test source tree. Only affects teams that have written custom test slice annotations; most applications won't have this at all.

WHAT YOU'LL SEE

```
$ mvn test-compile
[ERROR] /src/test/java/com/example/CustomTestAnnotation.java:[3,62]
  error: package org.springframework.boot.test.autoconfigure.properties does not exist
```

THE FIX

Import on custom test annotation

BEFORE

```
import org.springframework.boot.test.autoconfigure.properties.PropertyMapping;
```

AFTER

```
import org.springframework.boot.test.context.PropertyMapping;
```

HOW TO FIX

Update the import.

Replace `org.springframework.boot.test.autoconfigure.properties.PropertyMapping` with `org.springframework.boot.test.context.PropertyMapping`. This typically appears only in custom meta-annotations for test slices.

WATCH OUT

If you have a shared test-utilities library that defines custom slices, the fix must be applied there rather than in the consuming projects. A single unfixed library breaks all consumers.

FURTHER INFO

@PropertyMapping is used when authoring custom test slice annotations. It binds annotation attributes to Spring Boot auto-configuration properties. Several other test-autoconfigure classes moved in the same cleanup.

spring-jcl Removed

Tier 1 - Won't Build

Core Platform & Build

Impact: S

🔗 OpenRewrite

WHAT CHANGED

The `org.springframework:spring-jcl` module was removed in Spring Framework 7.0. Spring now depends directly on `commons-logging:commons-logging:1.3.0`, which gained native SLF4J and `java.util.logging` bridge support. Any POM that references `spring-jcl`, or excludes `commons-logging` in its favour, breaks the build.

WHY

Commons Logging 1.3 added the same SLF4J bridge that `spring-jcl` provided, making Spring's forked module redundant.

SCOPE CHECK

Search for `spring-jcl`, `jcl-over-slf4j`, and exclusions of `commons-logging` in all POM and Gradle files. All three are leftovers of the old logging bridge dance.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] Failed to execute goal on project my-service:
Could not resolve dependencies:
The following artifacts could not be resolved:
    org.springframework:spring-jcl:jar:7.0.0 (not found in central)
```

THE FIX

pom.xml — remove spring-jcl exclusion/inclusion

BEFORE

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <!-- Commons Logging 1.3.0 is pulled in transitively - no exclusion needed -->
</dependency>
```

pom.xml — remove explicit spring-jcl dependency

BEFORE

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-jcl</artifactId>
</dependency>
```

AFTER

```
<!-- spring-jcl no longer exists. Commons Logging 1.3.0 bridges to SLF4J natively. -->
```

HOW TO FIX

Remove spring-jcl references.

Delete any explicit `spring-jcl` dependency declarations and any `commons-logging` exclusions on Spring modules. Commons Logging 1.3.0 bridges to SLF4J out of the box.

Remove jcl-over-slf4j.

If you had `org.slf4j:jcl-over-slf4j` on the classpath, remove it. Commons Logging 1.3.0 provides the same bridge natively. Having both will cause a classpath conflict.

WATCH OUT

The classic "commons-logging exclusion + jcl-over-slf4j" pattern every Spring project used for a decade is now harmful: it excludes the Commons Logging 1.3.0 that Spring needs, then substitutes an older SLF4J bridge that may conflict. Remove both.

If you use a BOM or parent POM that applies global exclusions on commons-logging (common in large enterprises), that exclusion now breaks Spring. Update the corporate parent POM.

FURTHER INFO

A rare migration win: a decade of logging-bridge boilerplate can come out of your POMs.

[Spring-Break Demo](#) [Spring Boot 4.0 Migration Guide](#)

spring-retry Removed from BOM

Tier 1 - Won't Build

Spring Boot 4.0 removed spring-retry from its managed dependency BOM.

Declare an explicit version or the build fails.

Core Platform & Build

Impact: S

WHAT CHANGED

`org.springframework.retry:spring-retry` is no longer version-managed by the Spring Boot BOM. Any declaration without an explicit `<version>` tag fails because Maven/Gradle can't resolve it.

WHY

Spring Retry's release cadence diverged from Spring Boot's. Managing it in the BOM implied a level of compatibility testing that wasn't happening. Removing it from the BOM lets teams pin the version they test against.

SCOPE CHECK

Search for `spring-retry` in POM and Gradle files. Any declaration without an explicit version will fail. Also check for `@Retryable` and `@EnableRetry` annotations: these confirm the library is in use.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] 'dependencies.dependency.version' for org.springframework.retry:spring-retry:jar
is missing. @ line 42, column 17
[ERROR] Failed to execute goal on project my-service:
Could not resolve dependencies: Failed to collect dependencies at
org.springframework.retry:spring-retry:jar:RELEASE (no version)
```

THE FIX

pom.xml — add explicit version

BEFORE

```
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
  <version>2.0.11</version>
</dependency>
```

build.gradle — add explicit version

BEFORE

```
implementation 'org.springframework.retry:spring-retry'
```


AFTER

```
implementation 'org.springframework.retry:spring-retry:2.0.11'
```

HOW TO FIX

Add an explicit version.

Add `<version>2.0.11</version>` (or the latest compatible release) to your spring-retry dependency declaration. Check [Maven Central](#) for the latest version.

Import Spring Retry's own BOM.

If multiple modules use spring-retry, add its BOM to your `<dependencyManagement>` section so versions are managed in one place.

VERIFY

mvn dependency:tree shows aligned Spring Retry version and retry works

WATCH OUT

If you use `spring-cloud-starter-circuitbreaker-resilience4j`, it may have pulled in spring-retry transitively. With the BOM removal, the transitive version is no longer aligned. Pin it explicitly to avoid classpath surprises.

Spring Retry 2.x requires Spring Framework 6.0+. Since Boot 4.0 ships Framework 7.0, use Retry 2.0.x: older 1.x versions are incompatible.

FURTHER INFO

The library itself is unchanged and still maintained; only Boot's version management went away. See also: [retry-semantics](#), [retryable-transaction-order](#).

spring-break module: [spring-retry-removed](#) [Spring Boot 4.0 Migration Guide](#)

TIER 2

Tier 2 - Won't Start

4 cards

1. Deprecated RestTemplateBuilder APIs Removed
2. javax.annotation Removed
3. javax.inject Removed
4. Modular Auto-Configuration

Deprecated RestTemplateBuilder APIs Removed

Tier 2 - Won't Start

Deprecated `RestTemplateBuilder` methods are removed in Boot 4.0: code that compiled with warnings on 3.5 no longer compiles. Migrate to `RestClient`.

Core Platform & Build

Impact: S

WHAT CHANGED

Spring Boot 3.x deprecated several `RestTemplateBuilder` methods to signal the migration path to `RestClient` (introduced in Spring Framework 6.1). Boot 4.0 removes these deprecated methods entirely, following the standard deprecation-then-removal cycle.

WHY

`RestTemplate` is in maintenance mode. The modern replacement is `RestClient`, which offers a fluent, functional API and is the recommended HTTP client for new development in Spring Framework 6.x+. Removing the deprecated builder methods accelerates the migration.

SCOPE CHECK

Run `mvn compile` with `-Xlint:deprecation` on Boot 3.5 to find every deprecated API call before upgrading to 4.0. Pay particular attention to `RestTemplateBuilder` method chains: each deprecated call is a potential compile error on 4.0.

WHAT YOU'LL SEE

```
// Calling a deprecated builder method
RestTemplate rt = builder
    .additionalMessageConverters(new MappingJackson2HttpMessageConverter())
    .build();

// Boot 3.5: compiles with deprecation warning, runs fine.

// Boot 4.0: compile error.
error: cannot find symbol
  symbol: method additionalMessageConverters(MappingJackson2HttpMessageConverter)
```

THE FIX

Migrate from `RestTemplate` to `RestClient`

BEFORE

```
@Bean
public RestTemplate restTemplate(RestTemplateBuilder builder) {
    return builder
        .additionalMessageConverters(new MappingJackson2HttpMessageConverter())
        .build();
}
```

AFTER

```
@Bean
public RestClient restClient() {
    return RestClient.builder()
        .messageConverters(converters ->
            converters.add(new MappingJackson2HttpMessageConverter()))
        .build();
}
```

HOW TO FIX

Migrate to RestClient.

`RestClient` is the modern replacement. It offers a similar fluent builder API and is available from Spring Framework 6.1 onward (Spring Boot 3.2+). Migrating removes the deprecated call and future-proofs the HTTP client code.

If RestTemplate must be kept, remove the deprecated builder calls.

Construct a `RestTemplate` directly (via constructor or with a plain `RestTemplateBuilder.build()`) and configure message converters via `restTemplate.getMessageConverters().add(...)` after construction.

WATCH OUT

`RestTemplate` auto-configuration is also removed in Boot 4.0, a separate break from deprecated method removal. If you rely on an auto-configured `RestTemplate` bean rather than declaring one, see the `resttemplate-autoconfig` card.

FURTHER INFO

Affected methods include `additionalMessageConverters()`, `messageConverters()`, and `requestFactory()`. See also: `resttemplate-autoconfig`.

[spring-break module: deprecated-classes-removed](#) [Spring Boot 4.0 Migration Guide](#) [RestClient migration guide](#)

javax.annotation Removed

Tier 2 - Won't Start

Core Platform & Build

Impact: S

🔄 OpenRewrite

Spring Framework 7.0 stopped processing javax.annotation lifecycle annotations. @PostConstruct and @PreDestroy methods silently never fire. The code compiles and nothing warns you.

WHAT CHANGED

The `javax.annotation:javax.annotation-api` dependency is no longer on the Spring Boot classpath. Annotations `@javax.annotation.PostConstruct`, `@javax.annotation.PreDestroy`, and `@javax.annotation.Resource` are unavailable. The Jakarta equivalent is `jakarta.annotation:jakarta.annotation-api`, managed by the Boot BOM. See also: javax-inject-removed.

WHY

The Jakarta EE transition moved all javax APIs to jakarta. Spring Framework 7.0 completed this transition by dropping the legacy javax.annotation artifact.

SCOPE CHECK

Search for `javax.annotation` in all Java/Kotlin sources and build files. Pay particular attention to `@PostConstruct`: it appears in almost every service class with initialisation logic. Also check for `javax.annotation:javax.annotation-api` in POM files and Gradle build scripts.

WHAT YOU'LL SEE

```
// No compile error – javax.annotation-api is still downloadable.
// Failure is silent at runtime:

@PostConstruct
public void init() {
    this.ready = true; // never called on Boot 4.0
}

// Later in a test or endpoint:
java.lang.NullPointerException: Cannot invoke method because 'this.cache' is null
// or simply: assertions on initialisation state fail unexpectedly
```

THE FIX

Java imports

BEFORE

```
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
```

AFTER

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
```

pom.xml — if you had an explicit dependency

BEFORE

```
<dependency>
  <groupId>javax.annotation</groupId>
  <artifactId>javax.annotation-api</artifactId>
  <version>1.3.2</version>
</dependency>
```

AFTER

```
<dependency>
  <groupId>jakarta.annotation</groupId>
  <artifactId>jakarta.annotation-api</artifactId>
</dependency>
```

HOW TO FIX

Switch to jakarta.annotation.

Replace `javax.annotation` imports with `jakarta.annotation`. The annotations are identical: only the package name changed. The `jakarta.annotation-api` artifact is managed by the Spring Boot BOM so no version is needed.

Remove explicit dependency version.

If you declared `javax.annotation-api` explicitly in your POM, replace it with `jakarta.annotation-api` and omit the version.

VERIFY

App starts and `@PostConstruct` methods fire: confirm via log output or assert that initialisation state is set after context loads

WATCH OUT

`@PostConstruct` is skipped if the import is wrong: no error at startup, the method never runs. If your cache, connection pool, or scheduled task isn't initialising, check the import first.

`@Resource` (JSR-250 named injection) also lives in `javax.annotation`. If you use it for JNDI lookups or named bean injection, that import needs updating too.

FURTHER INFO

Driven by the Jakarta EE transition completed in Spring Framework 7.0. The `javax.annotation:javax.annotation-api` artifact still exists in Maven Central and still compiles. The break is at runtime, and `@Resource` injection stops working the same way.

spring-break module: `javax-annotation-removed` [Spring Boot 4.0 Migration Guide](#)

javax.inject Removed

Tier 2 - Won't Start

Core Platform & Build

Impact: S

OpenRewrite

Spring Framework 7.0 stopped processing `javax.inject` annotations. `@Inject` fields silently stay null. `@Named` beans may not be registered. The code compiles, the app starts, and nothing warns you.

WHAT CHANGED

The `javax.inject:javax.inject` dependency is no longer on the Spring Boot classpath. Annotations like `@javax.inject.Inject`, `@javax.inject.Named`, and `@javax.inject.Singleton` are unavailable. The Jakarta equivalent is `jakarta.inject:jakarta.inject-api`.

WHY

The Jakarta EE transition moved all `javax` APIs to `jakarta`. Spring Framework 7.0 completed this transition by dropping support for the legacy `javax.inject` artifact.

SCOPE CHECK

Search for `javax.inject` in all Java/Kotlin sources and build files. Every `import javax.inject` statement needs updating. Also check for `javax.inject:javax.inject` in POM files.

WHAT YOU'LL SEE

```
// No compile error – javax.inject:javax.inject still downloadable.
// Failure is silent at runtime:

@Named("orderService")
public class OrderService {
    @Inject
    private OrderRepository repo; // null on Boot 4.0 – never injected
}

// At call time:
java.lang.NullPointerException: Cannot invoke
    "com.example.OrderRepository.findById(Long)"
    because "this.repo" is null
```

THE FIX

Java imports

BEFORE

```
import javax.inject.Inject;
import javax.inject.Named;
```

AFTER

```
import jakarta.inject.Inject;
import jakarta.inject.Named;
```

pom.xml — if you had an explicit dependency

BEFORE

```
<dependency>
  <groupId>javax.inject</groupId>
  <artifactId>javax.inject</artifactId>
  <version>1</version>
</dependency>
```

AFTER

```
<dependency>
  <groupId>jakarta.inject</groupId>
  <artifactId>jakarta.inject-api</artifactId>
</dependency>
```

Or just use Spring annotations

BEFORE

```
@Named("orderService")
public class OrderService {
    @Inject
    private OrderRepository repo;
```

AFTER

```
@Service("orderService")
public class OrderService {
    @Autowired
    private OrderRepository repo;
```

HOW TO FIX

Switch to jakarta.inject.

Replace `javax.inject` imports with `jakarta.inject`. The annotations are identical: only the package name changed. The `jakarta.inject-api` artifact is managed by the Spring Boot BOM.

Switch to Spring annotations (recommended).

Replace `@Inject` with `@Autowired` and `@Named` with `@Component` / `@Service` / `@Qualifier`. This removes the dependency on the inject API entirely.

VERIFY

App starts and `@Inject` fields are non-null: confirm via a test that autowires a bean registered with `@Named`

WATCH OUT

If you're using `@Inject` for CDI compatibility (e.g., shared code between Spring and a Jakarta EE server), switch to `jakarta.inject`: it works in both environments. Don't switch to Spring-specific annotations if portability matters.

Constructor injection with `@Inject` on a single-constructor class can drop the annotation entirely. Spring auto-detects single-constructor injection since 4.3.

FURTHER INFO

Driven by the Jakarta EE transition completed in Spring Framework 7.0. The `javax.inject:javax.inject:1` artifact still exists in Maven Central and still compiles. The break is at runtime, and the first `NullPointerException` at call time

is your only clue.

spring-break module: javax-inject-removed

Spring Boot 4.0 Migration Guide

Modular Auto-Configuration

Tier 2 - Won't Start

Core Platform & Build

Impact: M

Boot 4.0 split auto-configuration into per-feature modules. Free beans (ObjectMapper, Validator) vanish without their starter. The app starts; NoSuchBeanDefinitionException hits at first use.

WHAT CHANGED

The `spring-boot-autoconfigure` jar bundled auto-configuration for every Spring Boot feature. Boot 4.0 splits it into focused modules, one per feature area, and each activates only when its corresponding starter is explicitly present. Adding `spring-boot-starter-web` no longer implicitly enables Jackson, validation, or actuator auto-configuration.

WHY

The monolithic jar meant any starter pulled in auto-configuration for hundreds of features the application never used. The split cuts startup time and memory, and makes dependencies explicit: you only pay for what you declare.

SCOPE CHECK

Run the app and scan for `NoSuchBeanDefinitionException` in startup logs. Also grep your code for `@Autowired ObjectMapper`, `@Autowired Validator`, and similar injections that relied on implicit auto-configuration. Check `@RestController` methods that return non-String types: these rely on Jackson message converters.

WHAT YOU'LL SEE

```
// App compiles and starts. Failure at first injection or endpoint call:

org.springframework.beans.factory.NoSuchBeanDefinitionException:
    No qualifying bean of type
    'com.fasterxml.jackson.databind.ObjectMapper' available

// Or from a @RestController trying to serialise a response:
org.springframework.http.converter.HttpMessageNotWritableException:
    No converter for [class com.example.OrderDto]
```

THE FIX

pom.xml — add starters that were previously implicit

BEFORE

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- Now required explicitly - no longer implicit via web starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-json</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

HOW TO FIX

Audit and add missing starters.

Run `mvn dependency:tree` and compare what was transitive on Boot 3.5 with what your Boot 4.0 build pulls in. Add an explicit `spring-boot-starter-*` dependency for every feature your application uses: `spring-boot-starter-json` for Jackson, `spring-boot-starter-validation` for Bean Validation, `spring-boot-starter-actuator` for Micrometer metrics.

Watch @RestController endpoints.

If a controller returns an object that Jackson used to serialise automatically, the endpoint now returns `406 Not Acceptable` or throws `HttpMessageNotWritableException` rather than an obvious startup error. Add `spring-boot-starter-json` to restore the message converter.

VERIFY

App starts and ObjectMapper, Validator, and other auto-configured beans are present: confirm with a test that autowires each one

WATCH OUT

The context loads fine; the missing bean surfaces at first injection or when a converter is needed. Endpoint smoke tests find the gaps fastest.

Each application hits a different subset of missing beans depending on the features it uses. There is no single fix: audit, using the Migration Guide's starter mapping table as a checklist.

`spring-boot-starter-web` still brings Jackson onto the *classpath* as a transitive compile dependency: the classes are present but never configured into beans. The app compiles with no import errors and no signal that anything is wrong.

FURTHER INFO

In Boot 3.x every starter dragged in the entire auto-configuration bundle, so features like Jackson, Bean Validation, and Micrometer configured themselves without a declared dependency.

spring-break module: modular-starters Spring Boot 4.0 Migration Guide Modularizing Spring Boot (Spring Blog)

TIER 3

Tier 3 - Behaves Differently

4 cards

1. Logback Default Charset Changed to UTF-8
2. @NullMarked Default — IDE/Compiler Null Safety
3. maxAttempts Now Means Retries, Not Total Attempts
4. @Retryable + @Transactional AOP Ordering Change

Logback Default Charset Changed to UTF-8

Tier 3 - Behaves Differently

Core Platform & Build

Impact: S

Boot 4.0 forces Logback file appenders to UTF-8. Non-ASCII characters appear garbled to tools expecting the platform's default encoding. No error: just wrong characters.

WHAT CHANGED

Spring Boot's default Logback configuration changed the charset for file appenders to UTF-8. Console appenders now use `Console#charset()` (the JVM's console charset, Java 17+) rather than the platform default. The change affects the default Logback XML provided by Spring Boot; custom Logback configurations that already specify an explicit charset are unaffected.

WHY

UTF-8 is the universal standard for text encoding. Platform-default encoding produced different bytes on different operating systems, making cross-platform log analysis unreliable.

SCOPE CHECK

Check `logback-spring.xml` and `logback.xml` for file appenders that do not specify a `<charset>`. Also audit downstream tools that specify the input encoding for log files: Splunk forwarders, Fluentd parsers, Elastic Filebeat configs.

WHAT YOU'LL SEE

```
# Boot 3.5 (platform encoding = Windows-1252):
2025-01-01 INFO  Order confirmed: €150.00 for Müller

# Boot 4.0 (UTF-8 forced, consumed by tool expecting Windows-1252):
2025-01-01 INFO  Order confirmed: â,-150.00 for MÃ¼ller

# Or if the terminal/tool can't decode UTF-8 sequences:
2025-01-01 INFO  Order confirmed: ?150.00 for M?ller
```

THE FIX

logback-spring.xml — if you need to override the default

BEFORE

```
<!-- Boot 3.5: no charset specified, platform default used for file appender -->
<appender name="FILE" class="ch.qos.logback.core.FileAppender">
  <file>app.log</file>
  <encoder>
    <pattern>%d{yyyy-MM-dd} %-5level %msg%n</pattern>
  </encoder>
</appender>
```

AFTER

```
<!-- Boot 4.0: UTF-8 is default; specify charset explicitly if you need platform encoding -->
<appender name="FILE" class="ch.qos.logback.core.FileAppender">
  <file>app.log</file>
  <encoder>
    <charset>UTF-8</charset>
    <pattern>%d{yyyy-MM-dd} %-5level %msg%n</pattern>
  </encoder>
</appender>
```

HOW TO FIX

Update log consumers to expect UTF-8.

If log files are consumed by external tools, log aggregators, or scripts, ensure those consumers are configured to read UTF-8. This is the correct long-term fix.

Restore platform encoding if needed.

If UTF-8 is not acceptable for your environment, add an explicit `<charset>` to your file appenders in `logback-spring.xml`. This is a short-term workaround; prefer updating consumers to UTF-8.

VERIFY

Log output contains correctly encoded characters (especially non-ASCII) in both console and file appenders after the charset change

WATCH OUT

This only affects file appenders. Console output uses the JVM console charset (usually controlled by the terminal), not forced to UTF-8.

Custom Logback configurations that already declare `<charset>UTF-8</charset>` explicitly are unaffected.

FURTHER INFO

Boot 3.5 used the platform's default encoding: often Cp1252 or GBK on Windows, sometimes ISO-8859-1 on Linux. Consumers built around those encodings will misread the new UTF-8 bytes in international text, currency symbols, and emoji.

spring-break module: logback-charset-change Spring Boot 4.0 Migration Guide

@Nullable Default — IDE/Compiler Null Safety

Tier 3 - Behaves Differently

Core Platform & Build

Impact: M

Spring Framework 7 applies `@Nullable` to every package. IDEs and null-checking tools now flag your existing null-passing code as warnings or errors.

WHAT CHANGED

Spring Framework 7 added `@Nullable` (from JSpecify) at the package level across the entire framework. This means all method parameters and return types are assumed `@NonNull` by default unless explicitly annotated `@Nullable`. IDEs, static analysis tools, and compile-time null checkers now see Spring's null contracts and flag violations.

WHY

Formal null contracts let tools catch `NullPointerException` bugs at compile time instead of runtime. Kotlin interop also improves: Kotlin's type system can now correctly infer nullability from Spring APIs.

SCOPE CHECK

Open your project in IntelliJ or VS Code with Java support and look at the new inspection warnings. Any code that passes `null` to a Spring API parameter, or uses a Spring API return value without a null check, will be flagged. The volume depends on how much Spring API surface your code touches directly.

WHAT YOU'LL SEE

```
// Your existing code – unchanged, worked fine in Boot 3.5
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
String name = ctx.getEnvironment().getProperty("app.name");
int length = name.length(); // ← IDE warning: name may be null

// IntelliJ inspection results after upgrade
WARNING: Method invocation 'length' may produce NullPointerException
        at MyService.java:42
WARNING: Passing 'null' argument to parameter annotated as @NonNull
        at MyConfig.java:18

// If using -Werror or NullAway/ErrorProne
$ mvn compile
[ERROR] MyService.java:[42,24] error: [NullAway] dereferencing expression
name, which is @Nullable
[ERROR] MyConfig.java:[18,35] error: [NullAway] passing @Nullable parameter
where @NonNull is required
```

THE FIX

Handle nullable return from `getProperty()`

BEFORE

```
String name = env.getProperty("app.name");
return name.toUpperCase();
```

AFTER

```
String name = env.getProperty("app.name");
if (name == null) {
    throw new IllegalStateException("app.name not configured");
}
return name.toUpperCase();
```

Or use the non-null variant

BEFORE

```
String name = env.getProperty("app.name");
```

AFTER

```
String name = env.getRequiredProperty("app.name");
```

Suppress warnings during migration

BEFORE

```
// (no null warnings in Boot 3.5)
```

AFTER

```
// Temporarily suppress in build.gradle
tasks.withType(JavaCompile) {
    options.compilerArgs += ['-Xlint:-nullness']
}
```

HOW TO FIX

Fix null handling in your code (recommended).

Follow the IDE warnings and add null checks, use `Optional`, or switch to non-null API variants like `getRequiredProperty()`. These are real NPE bugs that were always there; the framework now surfaces them.

Suppress during migration.

If the volume of warnings is too high to fix at once, suppress null-checking warnings at the compiler level and create a backlog to address them incrementally. Don't leave the suppression in place permanently.

VERIFY

No new `NullPointerException` at boundaries where nulls were OK before

WATCH OUT

If your CI pipeline uses `-Werror`, `NullAway`, `ErrorProne`, or `Checker Framework`, the new null annotations turn warnings into build failures. The framework's null contracts are now visible to your tools; unchanged code becomes a broken build.

Kotlin projects are affected differently. Kotlin's compiler uses these annotations to infer platform types. A Spring method that returned `String!` (platform type, nullable unknown) now returns `String` (non-null) or `String?` (nullable). Kotlin code that didn't handle nullability may now fail to compile.

FURTHER INFO

Runtime behaviour is unchanged: the impact lands entirely in compilers, IDEs, and static analysis tools.

Spring Boot 4.0 Migration Guide JSpecify @NullMarked

maxAttempts Now Means Retries, Not Total Attempts

Tier 3 - Behaves Differently

Core Platform & Build

Impact: M

`@Retryable(maxAttempts = 3)` now performs 3 retries (4 total calls) instead of 3 total attempts. Every retry site silently gains an extra call.

WHAT CHANGED

The `maxAttempts` parameter in `@Retryable` changed semantics. In Spring Retry 1.x (Boot 3.5) it meant total attempts including the initial call. In Boot 4.0 it means retries after the initial call. `maxAttempts = 3` now makes 4 calls.

WHY

The old naming was misleading: "max attempts" of 3 producing only 2 retries was a constant source of off-by-one bugs. The parameter name now matches what it counts, the number of retries.

SCOPE CHECK

Search for `@Retryable`, `RetryTemplate`, and any `maxAttempts` configuration in properties files. Every instance now makes one more call than before. For services with tight SLAs or rate-limited downstream APIs, this can cause cascading failures.

WHAT YOU'LL SEE

```
// Before (Spring Boot 3.5): 3 total attempts
INFO  Calling payment service (attempt 1/3)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 2/3)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 3/3)
ERROR Payment failed after 3 attempts

// After (Spring Boot 4.0): 1 initial + 3 retries = 4 total
INFO  Calling payment service (attempt 1/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 2/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 3/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 4/4)
ERROR Payment failed after 4 attempts

// Integration test failure
Expected invocation count: 3
Actual invocation count: 4
```

THE FIX

Fix: reduce maxAttempts by 1 to preserve old behaviour

BEFORE

```
@Retryable(maxAttempts = 3)
```

AFTER

```
@Retryable(maxAttempts = 2) // 1 initial + 2 retries = 3 total
```

Or use the new explicit parameter

BEFORE

```
@Retryable(maxAttempts = 3)
```

AFTER

```
@Retryable(retries = 2) // clearer: 2 retries after initial call
```

RetryTemplate configuration

BEFORE

```
RetryTemplate template = RetryTemplate.builder()  
    .maxAttempts(3)  
    .build();
```

AFTER

```
RetryTemplate template = RetryTemplate.builder()  
    .maxAttempts(2) // was 3 - now means retries, not total  
    .build();
```

HOW TO FIX

Subtract 1 from every maxAttempts value.

Search for all `@Retryable` annotations and `RetryTemplate` configurations. Reduce each `maxAttempts` value by 1 to preserve the original total attempt count.

Audit retry budgets.

The extra attempt may be acceptable or even desirable. Review each retry site and decide whether the new count is correct. Document the intended behaviour either way.

VERIFY

Retried operations use correct backoff timing and max attempts

WATCH OUT

The extra retry is invisible in logs unless you count carefully. Exponential backoff stretches the delay:
`maxAttempts = 5` with 2-second backoff now takes 62 seconds to fail instead of 30.

If your downstream service has rate limiting, the extra call per failure scenario may push you over the limit. Multiply the extra attempt by your error rate to estimate the impact.

FURTHER INFO

Driven by Spring Retry 2.x (upstream of Spring Boot 4.0). Spring Retry also left the Boot BOM in 4.0, so it needs an explicit version. See also: [spring-retry-bom](#), [retryable-transaction-order](#).

[spring-break module: retry-semantic-change](#) [Spring Framework Resilience docs](#)

@Retryable + @Transactional AOP Ordering Change

Tier 3 - Behaves Differently

Core Platform & Build

Impact: M

The default AOP advice ordering flipped: retry now wraps outside the transaction. A retried method gets a fresh transaction each attempt instead of retrying inside the same one.

WHAT CHANGED

The default ordering of Spring AOP advice changed so that `@Retryable` has higher precedence (runs first) than `@Transactional`. The retry logic now wraps the transaction, so each retry attempt runs in its own transaction.

WHY

Retrying outside the transaction is safer: a failed transaction rolls back fully before the next attempt, avoiding stale persistence context and dirty session state. The old default caused subtle Hibernate session corruption when retrying inside a failed transaction.

SCOPE CHECK

Search for any method annotated with both `@Retryable` and `@Transactional`. Each one has silently changed its transactional behaviour. Also check for `@Retryable` methods that call `@Transactional` methods on the same bean (self-invocation doesn't go through the proxy).

WHAT YOU'LL SEE

```
// Before (Spring Boot 3.5) – retry inside transaction
DEBUG Opening transaction
DEBUG Attempt 1: inserting order
WARN Optimistic lock failure, retrying...
DEBUG Attempt 2: inserting order
DEBUG Attempt 2: success
DEBUG Committing transaction (1 commit total)

// After (Spring Boot 4.0) – retry outside transaction
DEBUG Opening transaction #1
DEBUG Attempt 1: inserting order
WARN Optimistic lock failure
DEBUG Rolling back transaction #1
DEBUG Opening transaction #2
DEBUG Attempt 2: inserting order
DEBUG Attempt 2: success
DEBUG Committing transaction #2

// Symptom: partial writes visible between retries
// Symptom: retry sees stale data from previous transaction
```

THE FIX

Force retry inside transaction (old behaviour)

BEFORE

```
@Retryable(maxAttempts = 3)
@Transactional
public void placeOrder(Order order) {
```

AFTER

```
@Retryable(maxAttempts = 3)
@Transactional
@Order(Ordered.HIGHEST_PRECEDENCE) // force tx to wrap retry
public void placeOrder(Order order) {
```

Or set global ordering in configuration

BEFORE

```
# (default ordering in Boot 3.5: tx outside, retry inside)
```

AFTER

```
@EnableRetry(order = Ordered.LOWEST_PRECEDENCE)
// Ensures retry runs inside the transaction
```

Best practice: separate the concerns

BEFORE

```
@Retryable @Transactional
public void placeOrder(Order order) { ... }
```

AFTER

```
// Outer bean: handles retry
@Retryable(maxAttempts = 3)
public void placeOrderWithRetry(Order order) {
    orderService.placeOrder(order);
}
// Inner bean: handles transaction
@Transactional
public void placeOrder(Order order) { ... }
```

HOW TO FIX

Separate retry and transaction into different beans (recommended).

Move `@Retryable` to an outer service and `@Transactional` to an inner service. This makes the ordering explicit, avoids AOP precedence surprises, and works the same regardless of framework defaults.

Set explicit advice ordering.

Use `@EnableRetry(order = ...)` and `@EnableTransactionManagement(order = ...)` to explicitly control which advice runs first. Document the intended order.

VERIFY

Retried transactional methods execute retries outside the transaction

WATCH OUT

If your retry logic depends on reading data written earlier in the same transaction, that data is now rolled back before the retry. The second attempt starts with a

Optimistic locking exceptions (`OptimisticLockException`) are now handled differently. Previously the retry happened inside the

clean slate, which may cause duplicate inserts or missing context.

same persistence context. Now each retry gets a fresh EntityManager, which is usually better but changes the behaviour.

FURTHER INFO

Driven by a Spring Framework 7.0 change in default AOP advice ordering. See also: [retry-semantics](#), [spring-retry-bom](#).

[Spring-Break Demo](#)

[Spring Boot 4.0 Migration Guide](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break